

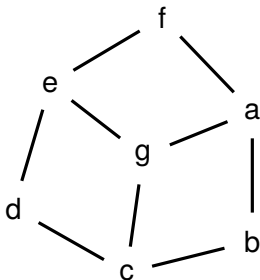
Graphs

Comp Sci 214, Spring 2025

Don't forget to check in on Youhere!

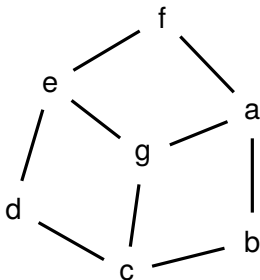
Graph Definitions Q&A

Let's put your knowledge to the test!



QUIZ: What is the degree of f?

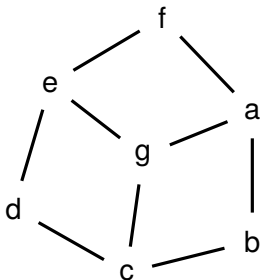
Let's put your knowledge to the test!



QUIZ: What is the degree of f? 2

QUIZ: What is the degree of g?

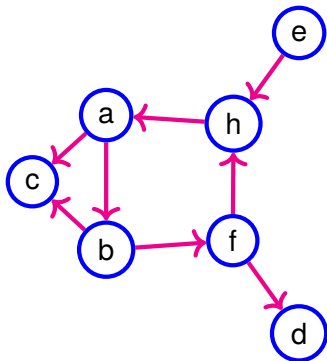
Let's put your knowledge to the test!



QUIZ: What is the degree of f? 2

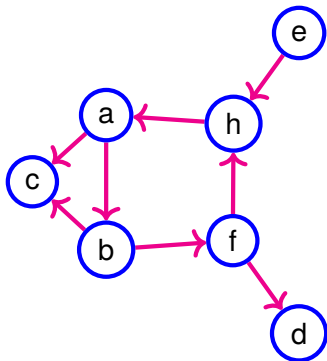
QUIZ: What is the degree of g? 3

Let's put your knowledge to the test!



QUIZ: Is E reachable from A?

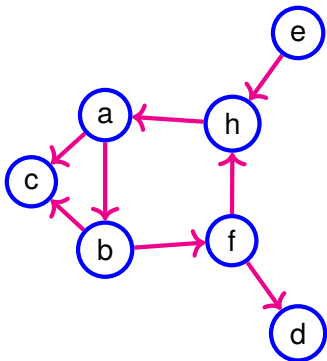
Let's put your knowledge to the test!



QUIZ: Is E reachable from A?

no

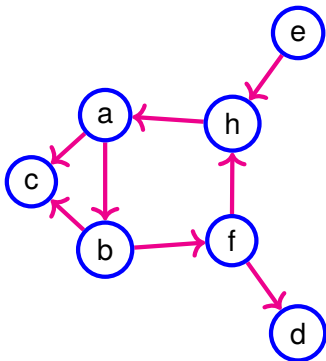
Let's put your knowledge to the test!



QUIZ: Is E reachable from A? no

QUIZ: Are A and B strongly connected?

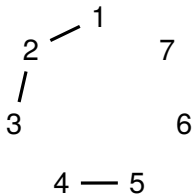
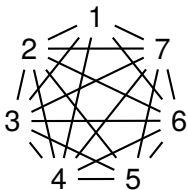
Let's put your knowledge to the test!



QUIZ: Is E reachable from A? no

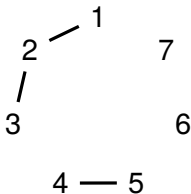
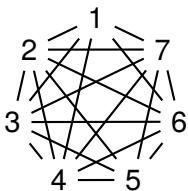
QUIZ: Are A and B strongly connected? yes

Let's put your knowledge to the test!



QUIZ: Which is the denser graph, left or right?

Let's put your knowledge to the test!

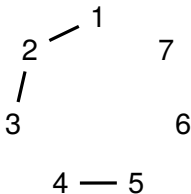
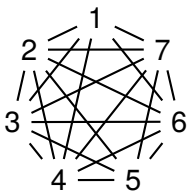


QUIZ: Which is the denser graph, left or right? left

QUIZ: What is the max # of edges for a graph with n nodes?

A: $\mathcal{O}(\log n)$ **B:** $\mathcal{O}(n)$ **C:** $\mathcal{O}(n^2)$ **D:** $\mathcal{O}(2^n)$

Let's put your knowledge to the test!



QUIZ: Which is the denser graph, left or right? left

QUIZ: What is the max # of edges for a graph with n nodes?

A: $\mathcal{O}(\log n)$ **B:** $\mathcal{O}(n)$ **C:** $\mathcal{O}(n^2)$ **D:** $\mathcal{O}(2^n)$

Stop! Question time!

Before we move on to graph ADT(s)...

Anything unclear so far?

Anything I missed?

Programming with graphs

Different kinds of graphs, different ADTs!

Two axes along which we can classify graphs:

- Directed vs undirected
- Weighted vs unweighted

Different kinds of graphs

→ different abstract values and operations

→ different ADTs!

Different kinds of graphs, different ADTs!

Two axes along which we can classify graphs:

- Directed vs undirected
- Weighted vs unweighted

Different kinds of graphs

→ different abstract values and operations

→ different ADTs!

Four combinations → four ADTs!

- Unweighted Undirected Graphs (UU)
- Unweighted Directed Graphs (UD)
- Weighted Undirected Graphs (WU)
- Weighted Directed Graphs (WD)

Today: #1, #2, and #4

Homework 4: #3

An unweighted undirected graph ADT

Abstract values: look like (V, E) (edges: sets of 2 vertices)

We will use the natural numbers $0 \dots n - 1$ as our vertices.

Operations:

```
interface UUGRAPH:
  def new_vertex(self) -> nat?
  def add_edge(self, u: nat?, v: nat?) -> NoneC
  def has_edge?(self, u: nat?, v: nat?) -> bool?
  def get_vertices(self) -> SetC[nat?]
  def get_neighbors(self, v: nat?) -> SetC[nat?]
```

Invariants:

- $V = \{0, 1, \dots, |V| - 1\}$
- $\bigcup E \subseteq V$

UU Graph ADT laws

$$\left\{ g = \boxed{(V, E)} \right\} g.\textit{new_vertex}() \Rightarrow n \left\{ g = \boxed{(V \cup \{n\}, E)} \wedge n = \max(V) + 1 \right\}$$

UU Graph ADT laws

$$\left\{g = \boxed{(V, E)}\right\} g.\textit{new_vertex}() \Rightarrow n \left\{g = \boxed{(V \cup \{n\}, E)} \wedge n = \max(V) + 1\right\}$$

$$\left\{g = \boxed{(V, E)} \wedge n, m \in V\right\} g.\textit{add_edge}(n, m) \Rightarrow \textit{None} \left\{g = \boxed{(V, E \cup \{\{n, m\}\})}\right\}$$

UU Graph ADT laws

$$\left\{ g = \boxed{(V, E)} \right\} g.\text{new_vertex}() \Rightarrow n \left\{ g = \boxed{(V \cup \{n\}, E)} \wedge n = \max(V) + 1 \right\}$$

$$\left\{ g = \boxed{(V, E)} \wedge n, m \in V \right\} g.\text{add_edge}(n, m) \Rightarrow \text{None} \left\{ g = \boxed{(V, E \cup \{\{n, m\}\})} \right\}$$

$$\left\{ g = \boxed{(V, E)} \wedge \{n, m\} \in E \right\} g.\text{has_edge?}(n, m) \Rightarrow \text{True} \{ \}$$

$$\left\{ g = \boxed{(V, E)} \wedge \{n, m\} \notin E \right\} g.\text{has_edge?}(n, m) \Rightarrow \text{False} \{ \}$$

UU Graph ADT laws

$$\{g = \boxed{(V, E)}\} \text{ g.new_vertex() } \Rightarrow n \{g = \boxed{(V \cup \{n\}, E)} \wedge n = \max(V) + 1\}$$

$$\{g = \boxed{(V, E)} \wedge n, m \in V\} \text{ g.add_edge}(n, m) \Rightarrow \text{None} \{g = \boxed{(V, E \cup \{\{n, m\}\})}\}$$

$$\{g = \boxed{(V, E)} \wedge \{n, m\} \in E\} \text{ g.has_edge?}(n, m) \Rightarrow \text{True} \{\}$$

$$\{g = \boxed{(V, E)} \wedge \{n, m\} \notin E\} \text{ g.has_edge?}(n, m) \Rightarrow \text{False} \{\}$$

$$\{g = \boxed{(V, E)}\} \text{ g.get_vertices() } \Rightarrow V \{\}$$

$$\{g = \boxed{(V, E)}\} \text{ g.get_neighbors}(n) \Rightarrow \{m \in V : \{m, n\} \in E\} \{\}$$

An unweighted directed graph ADT

Abstract values: look like (V, E) (E set of pairs of vertices)

Operations:

interface UDGRAPH:

```
def new_vertex(self) -> nat?
def add_edge(self, src: nat?, dst: nat?) -> NoneC
def has_edge?(self, src: nat?, dst: nat?) -> bool?
def get_vertices(self) -> SetC[nat?]
def get_succs(self, v: nat?) -> SetC[nat?]
def get_preds(self, v: nat?) -> SetC[nat?]
```

Order of arguments matters for add_edge and has_edge.
get_neighbors is split in two: get_succs and get_preds.

Invariants:

- $V = \{0, 1, \dots, |V| - 1\}$
- $\forall (v, u) \in E. v \in V \wedge u \in V$

UD Graph ADT laws, differences

$$\left\{ g = \boxed{(V, E)} \wedge n, m \in V \right\} \text{ *g.add_edge(n, m)* } \Rightarrow \text{None} \left\{ g = \boxed{(V, E \cup \{(n, m)\})} \right\}$$

Edges are directed: sets $\{n, m\}$ become pairs (n, m)

UD Graph ADT laws, differences

$$\left\{ g = \boxed{(V, E)} \wedge n, m \in V \right\} g.add_edge(n, m) \Rightarrow \text{None} \left\{ g = \boxed{(V, E \cup \{(n, m)\})} \right\}$$

Edges are directed: sets $\{n, m\}$ become pairs (n, m)

$$\left\{ g = \boxed{(V, E)} \wedge (n, m) \in E \right\} g.has_edge?(n, m) \Rightarrow \text{True} \{ \}$$

Order now matters: edge (n, m) is not the same as (m, n)

UD Graph ADT laws, differences

$$\left\{ g = \boxed{(V, E)} \wedge n, m \in V \right\} g.add_edge(n, m) \Rightarrow \text{None} \left\{ g = \boxed{(V, E \cup \{(n, m)\})} \right\}$$

Edges are directed: sets $\{n, m\}$ become pairs (n, m)

$$\left\{ g = \boxed{(V, E)} \wedge (n, m) \in E \right\} g.has_edge?(n, m) \Rightarrow \text{True} \{ \}$$

Order now matters: edge (n, m) is not the same as (m, n)

$$\left\{ g = \boxed{(V, E)} \right\} g.get_succs(n) \Rightarrow \{m \in V : (n, m) \in E\} \{ \}$$

$$\left\{ g = \boxed{(V, E)} \right\} g.get_preds(n) \Rightarrow \{m \in V : (m, n) \in E\} \{ \}$$

get_neighbors now split in two, source vs destination matters.

A weighted directed graph ADT

Abstract values: look like (V, E, w) (w maps edges to weights)

Operations:

```
let weight? = OrC(num?, inf)
```

```
interface WDGRAPH:
```

```
  def new_vertex(self) -> nat?
```

```
  def set_edge(self, src: nat?, w: weight?,  
                dst: nat?) -> NoneC
```

```
  def get_edge(self, src: nat?, dst: nat?) -> weight?
```

```
  def get_vertices(self) -> SetC[nat?]
```

```
  def get_succs(self, v: nat?) -> SetC[nat?]
```

```
  def get_preds(self, v: nat?) -> SetC[nat?]
```

We'll use a weight of ∞ to represent absent edges. Infinitely far.

Avoids False/None as special case, can use $<$ or $>$ directly.

WD graph ADT laws, differences, Part 1

$$\left\{ g = \boxed{(V, E, w)} \wedge n, m \in V \wedge a < \infty \right\}$$

$g.set_edge(n, a, m) \Rightarrow \text{None}$

$$\left\{ g = \boxed{(V, E \cup \{(n, m)\}, w \cup \{(n, m) \mapsto a\})} \right\}$$

$$\left\{ g = \boxed{(V, E, w)} \wedge n, m \in V \right\}$$

$g.set_edge(n, \infty, m) \Rightarrow \text{None}$

$$\left\{ g = \boxed{(V, E \setminus \{(n, m)\}, w \setminus \{(n, m)\})} \right\}$$

WD graph ADT laws, differences, Part 2

$$\left\{ g = \boxed{(V, E, w)} \wedge (n, m) \in E \right\} g.\textit{get_edge}(n, m) \Rightarrow w(n, m) \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \wedge (n, m) \notin E \right\} g.\textit{get_edge}(n, m) \Rightarrow \infty \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \right\} g.\textit{get_vertices}(g) \Rightarrow V \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \right\} g.\textit{get_succs}(n) \Rightarrow \{m \in V : (n, m) \in E\} \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \right\} g.\textit{get_preds}(n) \Rightarrow \{m \in V : (m, n) \in E\} \quad \{\}$$

Variants

- **Fixed vs extensible set of vertices**
 - ▶ Here: can add new vertices
 - ▶ Homework 4: number of vertices is fixed at graph creation
- **Access to set of edges**
 - ▶ Here: have to get each vertex's edges
 - ▶ Homework 4: `get_all_edges` to get them all at once
- **Direct access to predecessors**
 - ▶ Vs indirect: iterating over all possible preds (i.e., all nodes)
- Etc.

A **lot** of variations in practice.

- Graph DSs are often custom-built for a given application
 - ▶ → can pick the operations the application needs!
 - ▶ <https://www.hillelwayne.com/post/graph-types/>
- But all follow the same idea! Vertices, edges, etc.
- Missing ops can (usually) be written in terms of the others.
 - ▶ Not always efficiently, though.

Stop! Question time!

Before we move on to generalizing our graphs...

Anything unclear so far?

Anything I missed?

I am not a number, I am a free man!

We've been using natural numbers as our vertices, which is great for simplicity.

But graphs that we care about may have other data as vertices!

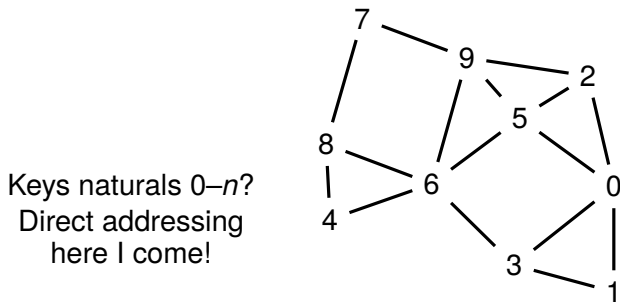
- Cities (with roads as edges)
- People (with friendships as edges)
- etc.

Turns out we can get the best of both worlds.

Any ideas how?

Dictionaries to the rescue!

Keep a dictionary mapping node ids to, e.g., people!


$$D = \{0:\textit{Alice}, 1:\textit{Bob}, 2:\textit{Charlie}, 3:\textit{Diane}, 4:\textit{Eve}, 5:\textit{Fred}, \dots\}$$

And reverse mapping to, e.g., find the names of Alice's friends.

If one ADT doesn't 100% solve your problem, try two!

Graph Data Structures

Graph Data Structures

We could use data structures we already know, but would be inefficient:

- Linked list of edges: `has_edge?` is $\mathcal{O}(|E|)$. Yuck.
 - ▶ I.e., linear search in our list of edges
 - ▶ $|E|$ = size of the set of edges = number of edges
- Hash table of edges: `get_neighbors` is $\mathcal{O}(|V|)$. Yuck.
 - ▶ I.e., try every vertex, see if there's an edge
 - ▶ $|V|$ = size of the set of vertices = number of vertices

Graph Data Structures

We could use data structures we already know, but would be inefficient:

- Linked list of edges: `has_edge?` is $\mathcal{O}(|E|)$. Yuck.
 - ▶ I.e., linear search in our list of edges
 - ▶ $|E|$ = size of the set of edges = number of edges
- Hash table of edges: `get_neighbors` is $\mathcal{O}(|V|)$. Yuck.
 - ▶ I.e., try every vertex, see if there's an edge
 - ▶ $|V|$ = size of the set of vertices = number of vertices
- **Note:** two measures of graph size: $|V|$ and $|E|$ instead of just one like most ADTs: n

Let's use some purpose-built data structures instead.

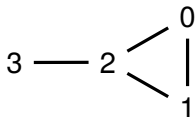
Graph Data Structures

There are two common data structures to represent graphs when programming:

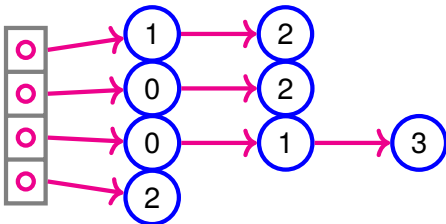
- *adjacency list*
- *adjacency matrix*

Others exist and can be used for special types of graphs, but the above two are the most basic.

Adjacency list



Array of lists of neighbors (or successors) for each vertex:

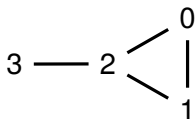


Can think of as a dictionary from node # to list of neighbors.

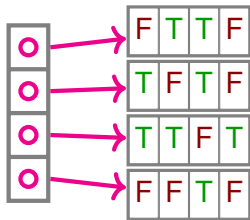
Keys are $0 \dots n - 1 \rightarrow$ direct addressing!

If graph is undirected, will be symmetric.

Adjacency matrix

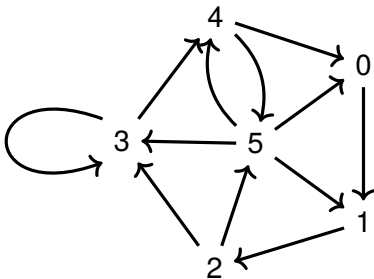


Store a $|V|$ -by- $|V|$ matrix of booleans indicating where edges are present:



Matrix usually represented as an array of arrays.

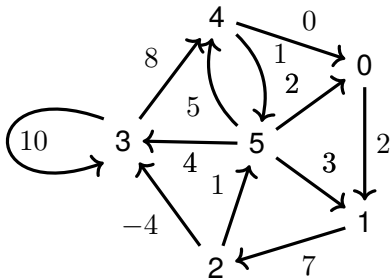
A directed adjacency matrix example



	0	1	2	3	4	5
0	F	T	F	F	F	F
1	F	F	T	F	F	F
2	F	F	F	T	F	T
3	F	F	F	T	T	F
4	T	F	F	F	F	T
5	T	T	F	T	T	F

Directed → not necc. symmetric!

With weights



	0	1	2	3	4	5
0	∞	2	∞	∞	∞	∞
1	∞	∞	7	∞	∞	∞
2	∞	∞	∞	-4	∞	1
3	∞	∞	∞	10	8	∞
4	0	∞	∞	∞	∞	1
5	2	3	∞	4	5	∞

Can store weights
with adj. lists too
(see chapter 12)

Asymptotic Space Complexity

We compared DS *operations* in terms of *time* complexity

- I.e., how time grows in relation to (some aspect of) the input

Same principles apply to the *space* required by *representations*

Example: have a set of n numbers, of which we (only) want to know the maximum element:

- Can store the whole set in a vector: space $\mathcal{O}(n)$
- Can store only the maximum element itself: space $\mathcal{O}(1)$

Asymptotic Space Complexity

We compared DS *operations* in terms of *time* complexity

- I.e., how time grows in relation to (some aspect of) the input

Same principles apply to the *space* required by *representations*

Example: have a set of n numbers, of which we (only) want to know the maximum element:

- Can store the whole set in a vector: space $\mathcal{O}(n)$
- Can store only the maximum element itself: space $\mathcal{O}(1)$

Wasn't relevant with earlier DSs: they all needed $\mathcal{O}(n)$ space!

E.g., a list with $2\times$ as many elements needs $2\times$ the space.

Asymptotic Space Complexity

We compared DS *operations* in terms of *time* complexity

- I.e., how time grows in relation to (some aspect of) the input

Same principles apply to the *space* required by *representations*

Example: have a set of n numbers, of which we (only) want to know the maximum element:

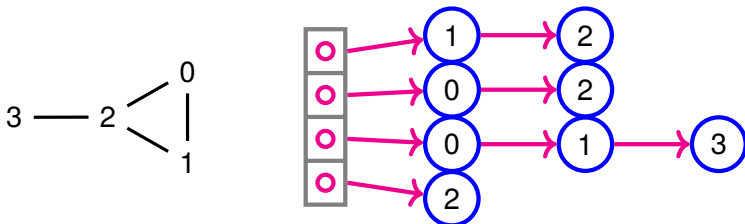
- Can store the whole set in a vector: space $\mathcal{O}(n)$
- Can store only the maximum element itself: space $\mathcal{O}(1)$

We can also speak of the space complexity of an operation.

Means: the auxiliary space the op needs aside from its input.

E.g., $\mathcal{O}(1)$ for in-place sorting, $\mathcal{O}(n)$ for out-of-place

Space Comparison: Adjacency List



Adjacency list: list for each vertex

- $|V|$ -element vector (“spine”)
- Total length of all the lists (“ribs”) = # of edges (or $2\times$)
 - ▶ 4 edges $\rightarrow$ 8 list elements total
- $\mathcal{O}(|V| + |E|)$

Space Comparison: Adjacency Matrix

	0	1	2	3	4	5
0	∞	2	∞	∞	∞	∞
1	∞	∞	7	∞	∞	∞
2	∞	∞	∞	-4	∞	1
3	∞	∞	∞	10	8	∞
4	0	∞	∞	∞	∞	1
5	2	3	∞	4	5	∞

Adjacency matrix: $|V|$ by $|V|$ matrix

- Matrix size only determined by $|V|$
- Regardless of how many edges are actually present
- $\mathcal{O}(|V|^2)$

Space Comparison: Adjacency Matrix

	0	1	2	3	4	5
0	∞	2	∞	∞	∞	∞
1	∞	∞	7	∞	∞	∞
2	∞	∞	∞	-4	∞	1
3	∞	∞	∞	10	8	∞
4	0	∞	∞	∞	∞	1
5	2	3	∞	4	5	∞

Adjacency matrix: $|V|$ by $|V|$ matrix

- Matrix size only determined by $|V|$
- Regardless of how many edges are actually present
- $\mathcal{O}(|V|^2)$

Q: For a sparse graph, which is more space-efficient?

Space Comparison: Adjacency Matrix

	0	1	2	3	4	5
0	∞	2	∞	∞	∞	∞
1	∞	∞	7	∞	∞	∞
2	∞	∞	∞	-4	∞	1
3	∞	∞	∞	10	8	∞
4	0	∞	∞	∞	∞	1
5	2	3	∞	4	5	∞

Adjacency matrix: $|V|$ by $|V|$ matrix

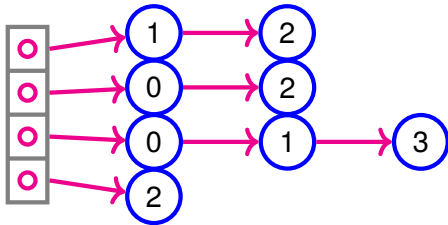
- Matrix size only determined by $|V|$
- Regardless of how many edges are actually present
- $\mathcal{O}(|V|^2)$

Q: For a sparse graph, which is more space-efficient?

A: Adjacency list. Sparse $\rightarrow |E|$ is much smaller than $|V^2|$

Even for dense, list no worse than matrix; matrix never better.

Time Comparison

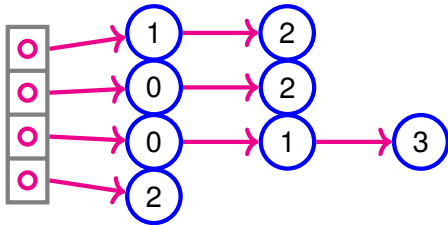


	0	1	2	3
0	F	T	T	F
1	T	F	T	F
2	T	T	F	T
3	F	F	T	F

add/set_edge and *get/has_edge*?

- **List:** worst case, traverse a whole rib: $\mathcal{O}(d)$
- **Matrix:** two array accesses: $\mathcal{O}(1)$

Time Comparison

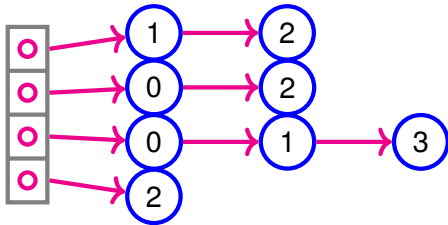


	0	1	2	3
0	F	T	T	F
1	T	F	T	F
2	T	T	F	T
3	F	F	T	F

get_succs

- **List:** copy one whole rib: $\mathcal{O}(d)$
- **Matrix:** traverse one whole row: $\mathcal{O}(|V|)$

Time Comparison



	0	1	2	3
0	F	T	T	F
1	T	F	T	F
2	T	T	F	T
3	F	F	T	F

get_preds

- **List:** traverse *all* ribs: $\mathcal{O}(|V| + |E|)$
- **Matrix:** traverse one whole *column*: $\mathcal{O}(|V|)$

Overall: matrix generally more time efficient

Oh, but what if my graphs are trees?

Our two graph data structures work perfectly fine with trees!

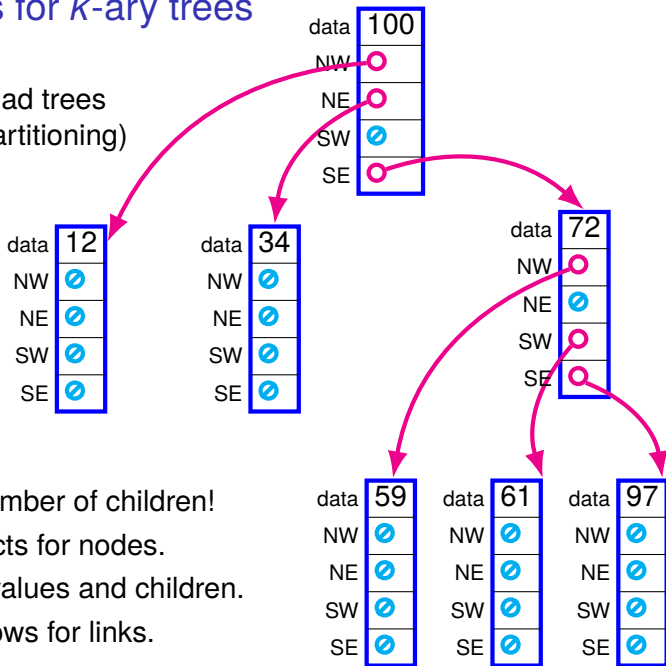
- After all, trees are a kind of graph!
- But there are better choices.
- More restrictions → can cut corners!
 - ▶ Cf. direct addressing vs general-purpose dictionaries

Let's look at two tree-specific data structures. More exist!

- See path trees in graph search lecture
- See binary heaps in priority queues lecture

Structs for k -ary trees

E.g., quad trees
(space partitioning)



Fixed number of children!

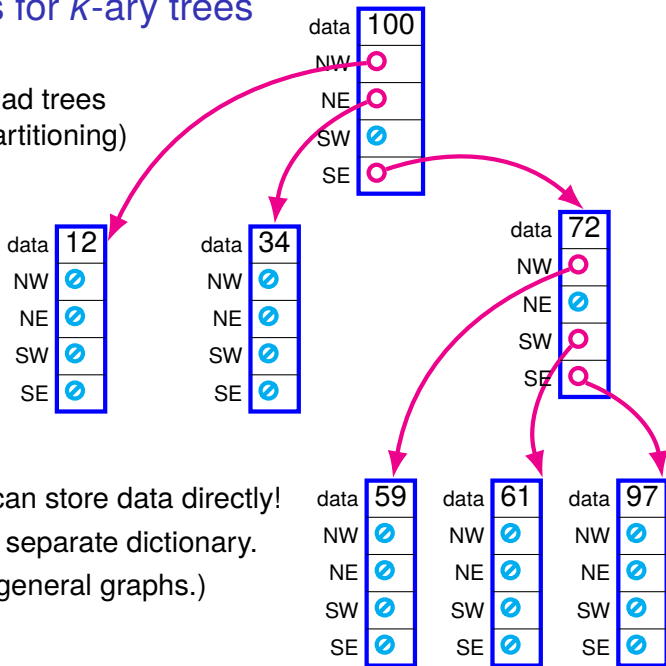
Structs for nodes.

Fields for values and children.

Arrows for links.

Structs for k -ary trees

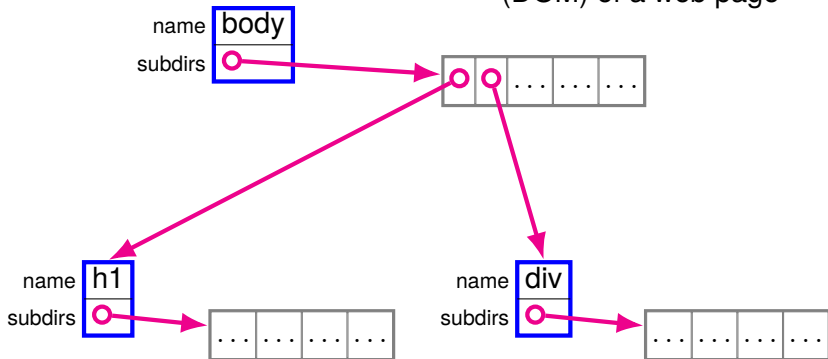
E.g., quad trees
(space partitioning)



With structs, can store data directly!
No need for separate dictionary.
(Unlike general graphs.)

Rose trees using arrays

E.g., the Document Object Model (DOM) of a web page



Variable number of children
→ need to use arrays! (or linked lists, or ...)

If we have time: general
Q&A for the midterm

Next time: graph search

Complete UU Graph ADT laws

$$\left\{ g = \boxed{(V, E)} \right\} g.\text{new_vertex}() \Rightarrow n \left\{ g = \boxed{(V \cup \{n\}, E)} \wedge n = \max(V) + 1 \right\}$$

$$\left\{ g = \boxed{(V, E)} \wedge n, m \in V \right\} g.\text{add_edge}(n, m) \Rightarrow \text{None} \left\{ g = \boxed{(V, E \cup \{\{n, m\}\})} \right\}$$

$$\left\{ g = \boxed{(V, E)} \wedge \{n, m\} \in E \right\} g.\text{has_edge?}(n, m) \Rightarrow \text{True} \{ \}$$

$$\left\{ g = \boxed{(V, E)} \wedge \{n, m\} \notin E \right\} g.\text{has_edge?}(n, m) \Rightarrow \text{False} \{ \}$$

$$\left\{ g = \boxed{(V, E)} \right\} g.\text{get_vertices}() \Rightarrow V \{ \}$$

$$\left\{ g = \boxed{(V, E)} \right\} g.\text{get_neighbors}(n) \Rightarrow \{ m \in V : \{m, n\} \in E \} \{ \}$$

Complete UD Graph ADT laws

$$\{g = \boxed{(V, E)}\} \text{ g.new_vertex() } \Rightarrow n \{g = \boxed{((V \cup \{n\}, E))} \wedge n = \max(V) + 1\}$$

$$\{g = \boxed{(V, E)} \wedge n, m \in V\} \text{ g.add_edge}(n, m) \Rightarrow \text{None} \{g = \boxed{(V, E \cup \{(n, m)\})}\}$$

$$\{g = \boxed{(V, E)} \wedge (n, m) \in E\} \text{ g.has_edge?}(n, m) \Rightarrow \text{True} \{\}$$

$$\{g = \boxed{(V, E)} \wedge (n, m) \notin E\} \text{ g.has_edge?}(n, m) \Rightarrow \text{False} \{\}$$

$$\{g = \boxed{(V, E)}\} \text{ g.get_vertices() } \Rightarrow V \{\}$$

$$\{g = \boxed{(V, E)}\} \text{ g.get_succs}(n) \Rightarrow \{m \in V : (n, m) \in E\} \{\}$$

$$\{g = \boxed{(V, E)}\} \text{ g.get_preds}(n) \Rightarrow \{m \in V : (m, n) \in E\} \{\}$$

Complete WD graph ADT laws, Part 1

$$\left\{ g = \boxed{(V, E, w)} \right\}$$

$$g.\text{new_vertex}() \Rightarrow n$$

$$\left\{ g = \boxed{(V \cup \{n\}, E, w)} \wedge n = \max(V) + 1 \right\}$$

$$\left\{ g = \boxed{(V, E, w)} \wedge n, m \in V \wedge a < \infty \right\}$$

$$g.\text{set_edge}(n, a, m) \Rightarrow \text{None}$$

$$\left\{ g = \boxed{(V, E \cup \{(n, m)\}, w \cup \{(n, m) \mapsto a\})} \right\}$$

$$\left\{ g = \boxed{(V, E, w)} \wedge n, m \in V \right\}$$

$$g.\text{set_edge}(n, \infty, m) \Rightarrow \text{None}$$

$$\left\{ g = \boxed{(V, E \setminus \{(n, m)\}, w \setminus \{(n, m)\})} \right\}$$

Complete WD graph ADT laws, Part 2

$$\left\{ g = \boxed{(V, E, w)} \wedge (n, m) \in E \right\} g.get_edge(n, m) \Rightarrow w(n, m) \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \wedge (n, m) \notin E \right\} g.get_edge(n, m) \Rightarrow \infty \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \right\} g.get_vertices(g) \Rightarrow V \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \right\} g.get_succe(n) \Rightarrow \{m \in V : (n, m) \in E\} \quad \{\}$$

$$\left\{ g = \boxed{(V, E, w)} \right\} g.get_preds(n) \Rightarrow \{m \in V : (m, n) \in E\} \quad \{\}$$